NumPy Day 1: Arrays

All examples assume: `import numpy as np`

What I learned

Concept	Syntax	What it means
Import	`import numpy as np`	Loads NumPy. `np` is its nickname.
Make an array	`arr = np.array([1, 2, 3])`	One list goes inside the parentheses.
Type of the array	`type(arr)`	`numpy.ndarray`
Type of the elements	`arr.dtype`	`int64` = whole numbers, `float64` = decimals. No parentheses.
Array vs list	`list * 2` `arr * 2`	The list repeats. The array doubles every element.
One type per array	`np.array([1, 2, 3.5])`	Mixing whole numbers and decimals makes everything `float64`.
Number + list	`nums + 10`	Error for a list, works on every element for an array.
No loops needed	`prices_arr * 2`	One line replaces a whole `for` loop.
Division	`arr / 5`	Always gives decimals (`float64`).

Plain words: decimal means `float`, and whole number means `int`.

Things to remember

1. Square brackets alone make a list. Only `np.array(...)` makes an array.
2. `np.array([1, 2, 3])` has parentheses around the list. `np.array[1, 2, 3]` is wrong.
3. Keep the list name and the array name apart (`temps_c` is the list, `temps` is the array), and use the array in the math.
4. `dtype` has no parentheses, because it is data the array holds. `type(...)` does have them.
5. `x = print(...)` saves nothing, because `print` gives back nothing. Put the calculation straight after `=`.

Practice problems and solutions

These are the problems from this day with working solutions and the real output. Try each one yourself before reading the solution.

Problem 1 (easy)

Create a NumPy array called `nums` with the values `10, 20, 30, 40, 50`. Print the array, then its type, then `nums * 2`.

SOLUTION

```
import numpy as np

nums = np.array([10, 20, 30, 40, 50])
print(nums)
print(type(nums))
print(nums * 2)
```

OUTPUT

```
[10 20 30 40 50]
<class 'numpy.ndarray'>
[ 20  40  60  80 100]
```

What it taught: `type(nums)` tells you what the thing is (`numpy.ndarray`). `nums.dtype` tells you what kind of numbers are inside.

Problem 2 (easy)

Create a normal list `prices = [100, 250, 400]` and print `prices * 2`. Then make an array `prices_arr` from it and print `prices_arr * 2`. Add a comment in your own words saying what is different.

SOLUTION

```
import numpy as np

prices = [100, 250, 400]
print(prices * 2)

prices_arr = np.array(prices)
# a list repeats; an array multiplies every element
print(prices_arr * 2)
```

OUTPUT

```
[100, 250, 400, 100, 250, 400]
[200 500 800]
```

What it taught: Math on a list repeats it, but math on an array works on every element. That difference is why NumPy exists.

Problem 3 (medium)

Make three arrays: a from `[1, 2, 3]`, b from `[1.5, 2.5, 3.5]`, and c from `[1, 2, 3.5]`. Write a comment above each one guessing its `dtype`. Then print `a.dtype`, `b.dtype` and `c.dtype`, and print `c * 2`.

SOLUTION

```
import numpy as np

a = np.array([1, 2, 3])          # integer
b = np.array([1.5, 2.5, 3.5])    # decimal
c = np.array([1, 2, 3.5])        # decimal, because one decimal changes all

print(a.dtype, b.dtype, c.dtype)
print(c * 2)
```

OUTPUT

```
int64 float64 float64
[2. 4. 7.]
```

What it taught: An array holds one type for all its elements. Mixing whole numbers and decimals makes everything decimals.

Problem 4 (medium)

Start with a list `nums = [1, 2, 3]`. Inside a `try`/`except TypeError`, run `nums + 10` and print the result. In the `except`, print `"A list can't do this"`. Then make an array `nums_arr` from `nums` and print `nums_arr + 10`.

SOLUTION

```
import numpy as np

nums = [1, 2, 3]

try:
    print(nums + 10)
except TypeError:
    print("A list can't do this")

nums_arr = np.array(nums)
print(nums_arr + 10)
```

OUTPUT

```
A list can't do this
[11 12 13]
```

What it taught: Adding a number to a list is an error, but adding a number to an array just works on every element.

Problem 5 (hard)

Start with `prices = [100, 250, 400]`. Using a `for` loop and `.append()`, build a new list `doubled` with every price doubled and print it. Then make an array `prices_arr` and double it in one line, with no loop, and print it.

SOLUTION

```
import numpy as np

prices = [100, 250, 400]
doubled = []

for price in prices:
    doubled.append(price * 2)

print(doubled)

prices_arr = np.array(prices)
print(prices_arr * 2)
```

OUTPUT

```
[200, 500, 800]
[200 500 800]
```

What it taught: A list needs a loop to change every item, but an array does the same job in one line.

Problem 6 (hard)

`temps_c = [0, 20, 37, 100]` is a list of temperatures in Celsius. The formula to convert one to Fahrenheit is `C * 9 / 5 + 32`. Make an array `temps` from the list, convert every temperature in one line with no loop and save it as `temps_f`, then print `temps_f` and `temps_f.dtype`. Add a comment guessing why the `dtype` is not `int64`.

SOLUTION

```
import numpy as np

temps_c = [0, 20, 37, 100]

temps = np.array(temps_c)
# division with / always gives decimals, so the result is float64
temps_f = temps * 9 / 5 + 32

print(temps_f)
print(temps_f.dtype)
```

OUTPUT

```
[ 32.   68.   98.6 212. ]
float64
```

What it taught: One line of array math replaces a loop, and `/` always turns the result into decimals (`float64`).